9. Special Cron shortcuts

Cron also supports some predefined scheduling shortcuts.

Shortcut	Meaning
`@reboot`	Run when the cron service starts after boot
`@yearly`	Once every year
`@monthly`	Once every month
`@weekly`	Once every week
`@daily`	Once every day
`@hourly`	Once every hour

Examples:

```bash
@daily /home/akankshya/backup.sh
```

```bash
@reboot /home/akankshya/startup.sh
```

Remember: `@reboot` refers to the cron daemon/system startup event, not every time you open a terminal.

8. Real-world examples

Imagine you're managing a Linux server.

Example 1: Daily backup at 10 PM

```bash
0 22 * * * /home/akankshya/backup.sh
```

Example 2: Run Python script every morning at 8 AM

```bash
0 8 * * * /usr/bin/python3 /home/akankshya/script.py
```

Example 3: Delete temporary files every Sunday at midnight

```bash
0 0 * * 0 rm -f /home/akankshya/temp/*
```

Be careful: this deletes matching files permanently.

Example 4: Check server health every 5 minutes

```bash
*/5 * * * * /home/akankshya/health-check.sh
```

Example 5: Save a backup log every day at 11:30 PM

```bash
30 23 * * * /home/akankshya/backup.sh >> /home/akankshya/backup.log 2>&1
```

Here:

- `>>` appends output to the log.
- `2>&1` sends errors to the same log as normal output.

10. How to check whether Cron is running

Cron jobs need the Cron daemon (background service) to execute.

Check its status:

```bash
systemctl status cron
```

If it's running, you should see:

```
Active: active (running)
```

Other useful commands:

Command	Purpose
`sudo systemctl start cron`	Start Cron service
`sudo systemctl stop cron`	Stop Cron service
`sudo systemctl restart cron`	Restart Cron service
`sudo systemctl enable cron`	Enable automatic startup
`sudo systemctl status cron`	Check service status

Important for your WSL Ubuntu: Cron may not automatically start when WSL opens. If the service is stopped, try:

```bash
sudo service cron start
```

If you get `command not found`, install it:

```bash
sudo apt update
sudo apt install cron
```

You can then check:

```bash
systemctl status cron
```

11. How to see Cron logs

Suppose you created a Cron Job, but it isn't executing.

How do you troubleshoot it?

You can check Cron logs:

```bash
sudo grep CRON /var/log/syslog
```

This displays Cron-related entries from the system log, if available.

You can also check:

```bash
journalctl -u cron
```

This displays logs associated with the Cron service when systemd journal logging is available.

12. Important commands – Notebook revision table

Command	Purpose
`crontab -l`	List current user's Cron Jobs
`crontab -e`	Create or edit Cron Jobs
`crontab -r`	Remove all current user's Cron Jobs
`crontab -u user -l`	List another user's jobs
`systemctl status cron`	Check Cron service
`sudo service cron start`	Start Cron service
`sudo systemctl stop cron`	Stop Cron service
`sudo systemctl restart cron`	Restart Cron service
`sudo grep CRON /var/log/syslog`	Check Cron log entries
`journalctl -u cron`	View Cron service logs

Remember: `crontab -r` removes all jobs belonging to the current user. Don't run it casually!

```
* * * * *       # Every minute
*/5 * * * *     # Every 5 minutes
0 8 * * *       # Every day at 8 AM
0 22 * * *      # Every day at 10 PM
```

1. What is a file system?

Imagine your laptop has thousands of files:

- Photos
- Videos
- Documents
- C++ programs
- Python files
- Applications
- System files

A **file system** is the way an operating system organizes, stores, names, and manages files and directories on a storage device.

Think of it like a library:

- Books = Files
- Shelves = Directories (folders)
- Library organization system = File system

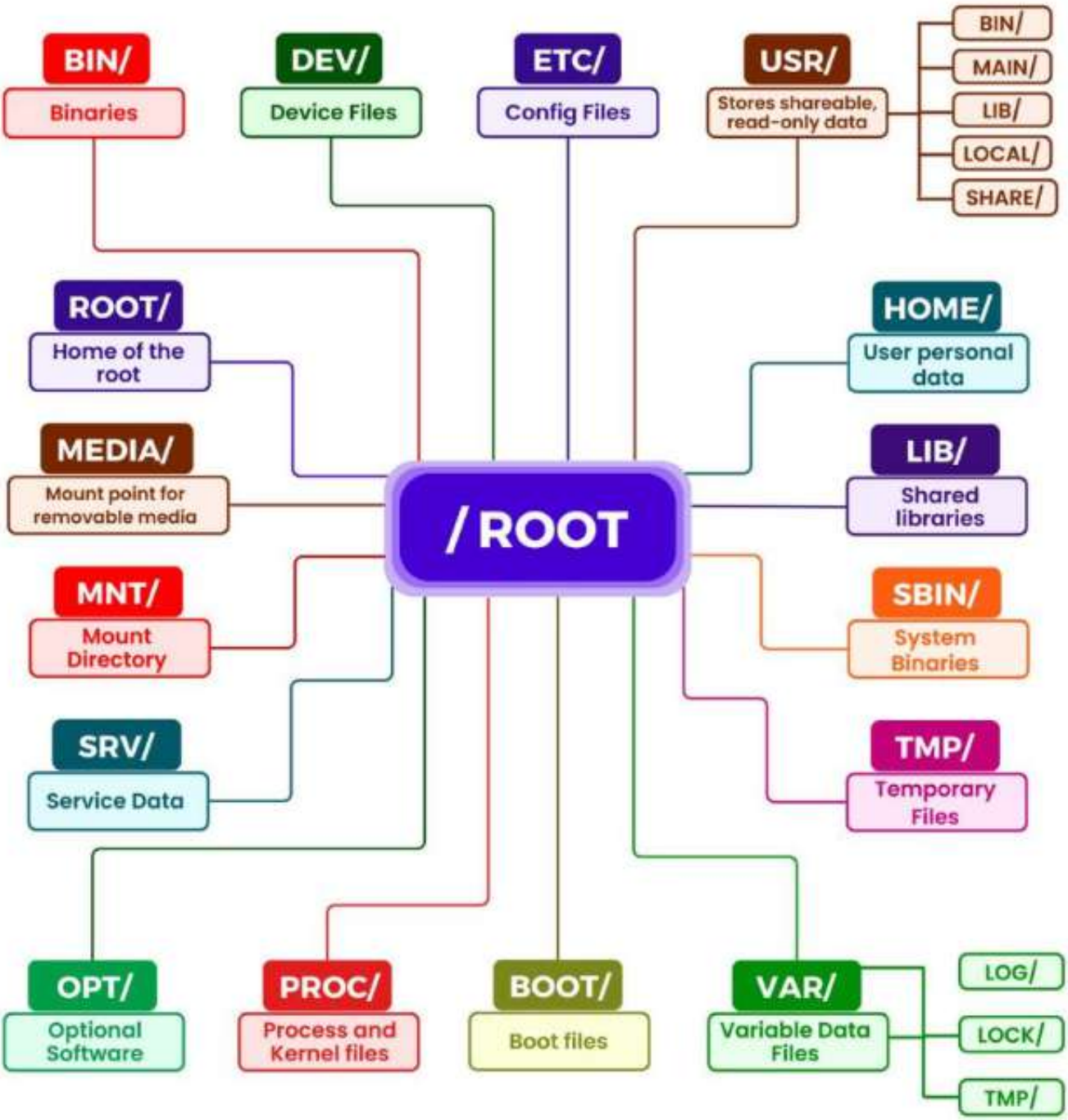
Without a file system, the operating system wouldn't have a structured way to locate and manage your data.

2. Linux file system structure

Linux organizes everything under one main directory called the **root directory**, represented by `/`.

Linux Directory Structure

WebMagic Informatica



Unlike Windows, where you commonly see drives such as `C:` and `D:`, Linux uses one unified directory tree.

For example:

Plain text

```
/
├── home
│   └── akankshya
│       ├── Documents
│       ├── Downloads
│       └── projects
│
├── etc
├── bin
├── boot
├── dev
├── media
├── mnt
├── root
├── tmp
├── usr
└── var
```

3. Important Linux directories

Directory	Purpose
`/`	Root directory; beginning of the entire filesystem
`/home`	Personal directories of normal users
`/root`	Home directory of the root user
`/etc`	System configuration files
`/bin`	Essential user commands and programs
`/sbin`	System administration commands
`/boot`	Files required for booting Linux
`/dev`	Device files
`/tmp`	Temporary files
`/var`	Frequently changing data, such as logs
`/usr`	Applications, libraries and shared resources
`/mnt`	Common mount point for temporarily mounted filesystems
`/media`	Common mount point for removable devices
`/opt`	Optional additional software

Important distinction:

- `/` = root of the entire Linux filesystem.
- `/root` = personal home directory of the root administrator.
- `/home/akankshya` = your normal user's home directory.

5. What is a mount point?

This is an important Linux concept.

Imagine you connect a USB drive to your computer.

Linux needs to make that drive accessible somewhere inside its directory tree.

That location is called a **mount point.**

For example:

```
Plain text
/mnt/usb
```

Once the USB filesystem is mounted there, you can access its files through `/mnt/usb`.

In simple words:

Mounting = Connecting a storage filesystem to a location in Linux's directory tree.

To see mounted filesystems:

```
Bash
mount
```

A more readable command:

```
Bash
findmnt
```

6. What are Linux filesystem types?

Different filesystems organize data differently.

Filesystem	Meaning / Usage
ext4	Common Linux filesystem
ext3	Older Linux filesystem
ext2	Older Linux filesystem
XFS	High-performance journaling filesystem
Btrfs	Modern filesystem with snapshots and other features
FAT32	Commonly used on USB drives
NTFS	Commonly used by Windows

For now, remember **ext4** because it is a widely used Linux filesystem.

7. Linux treats many things as files

You may hear the phrase:

"Everything is a file in Linux."

It means Linux provides a file-like interface for many resources, including devices and system information.

Some examples:

Type	Example	Meaning
Regular file	`notes.txt`	Stores ordinary data
Directory	`/home`	Contains directory entries
Device file	`/dev/null`	Represents a device-like interface
Symbolic link	`shortcut`	Points to another file or directory
FIFO	Named pipe	Used for communication between processes

8. Practical commands to remember

Command	Purpose
`pwd`	Show current directory
`ls /`	List root directory contents
`df -h`	Check filesystem disk space
`du -sh foldername`	Check folder size
`mount`	Show mounted filesystems
`findmnt`	Display mount information
`lsblk`	Show block storage devices
`file filename`	Identify a file's type